LOW-LEVEL DESIGN (LLD) DOCUMENT

Objective

The Low-Level Design (LLD) defines how each component of the RAG-based Customer Support Assistant is implemented internally. It focuses on individual modules, their responsibilities, and how they interact to process data and generate responses efficiently.

1. MODULE-LEVEL DESIGN

1.1 Document Processing Module

The Document Processing Module is responsible for loading and preparing the input PDF for further processing. It uses a PDF parsing library to extract text content and convert it into a structured format such as a list of documents. This module ensures that all relevant information from the knowledge base is captured accurately. It acts as the entry point of the data pipeline and passes the extracted text to the chunking module for further transformation.

1.2 Chunking Module

The Chunking Module takes the raw text from the document processing stage and divides it into smaller, manageable chunks. This module typically uses fixed-size chunking with optional overlap to preserve contextual continuity between segments. The purpose of this module is to improve retrieval accuracy and ensure that each chunk contains meaningful information. The output is a collection of text segments ready for embedding.

1.3 Embedding Module

The Embedding Module converts each text chunk into a vector representation using an embedding model. These embeddings capture the semantic meaning of the text and allow the system to perform similarity-based searches. This module is critical because it transforms textual data into a format that can be efficiently indexed and compared. The generated embeddings are forwarded to the vector storage module.

1.4 Vector Storage Module

The Vector Storage Module is responsible for storing and managing embeddings. In this system, ChromaDB is used to store vectors along with their associated metadata. This module

provides efficient indexing and similarity search capabilities, enabling fast retrieval of relevant data. It acts as the system's knowledge repository and supports persistent storage for reuse across sessions.

1.5 Retrieval Module

The Retrieval Module handles the process of fetching relevant information based on user queries. It converts the incoming query into an embedding and performs a similarity search in the vector store to identify the most relevant chunks. This module ensures that only the most contextually appropriate data is selected and passed to the query processing stage. It plays a key role in improving the accuracy and relevance of responses.

1.6 Query Processing Module

The Query Processing Module is responsible for generating the final response. It takes the user query and the retrieved context as input and sends them to a Large Language Model (LLM). The LLM processes this information and generates a natural language response. This module may also include prompt engineering techniques to guide the model in producing accurate and concise answers. It ensures that the output is user-friendly and contextually grounded.

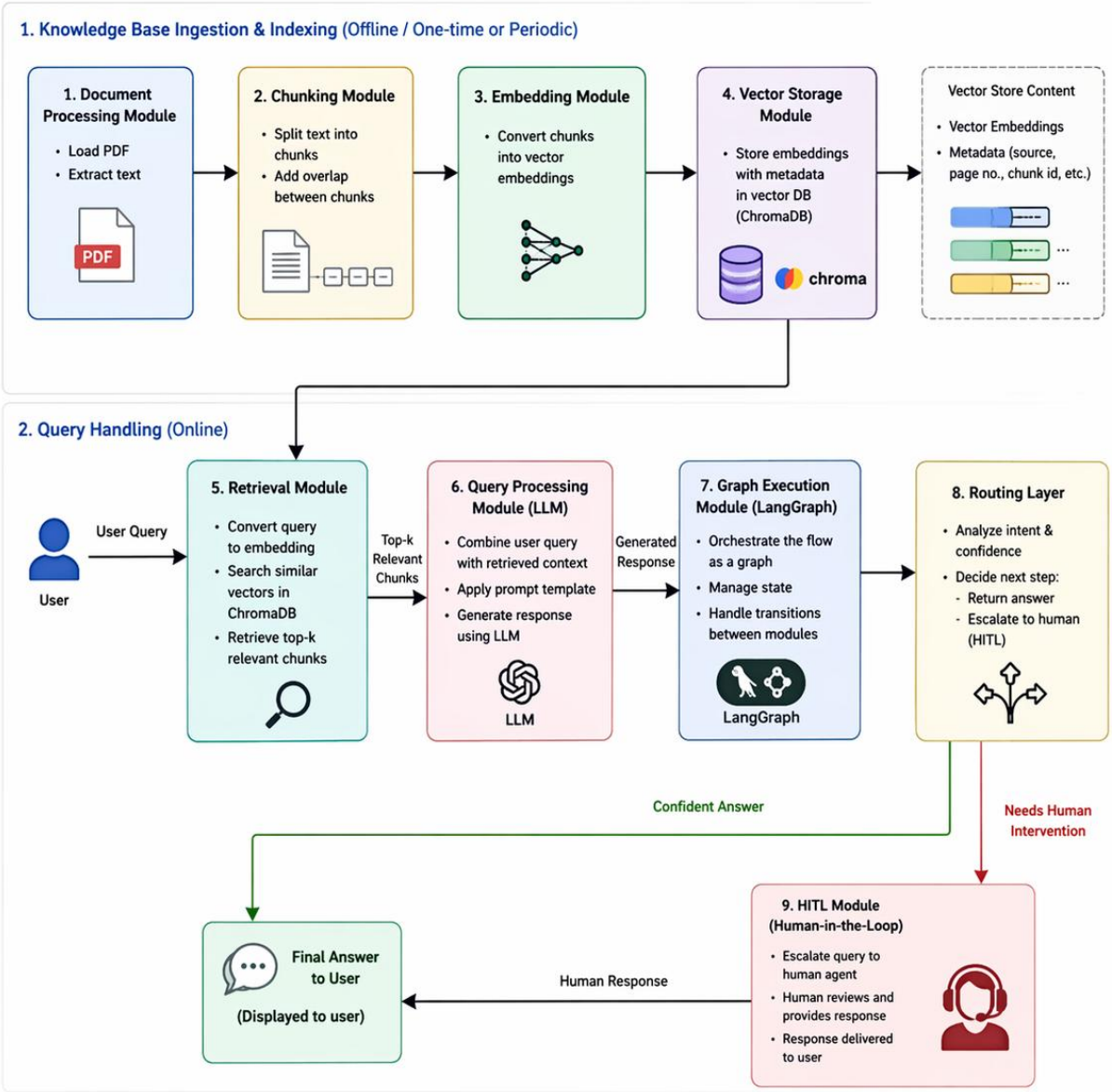
1.7 Graph Execution Module

The Graph Execution Module manages the overall workflow of the system. It is implemented using LangGraph, which allows the system to define nodes and transitions in a graph structure. This module controls the sequence of operations, including query handling, retrieval, and response generation. It also enables conditional routing, ensuring that the system can dynamically decide the next step based on the query type or system state.

1.8 HITL Module (Human-in-the-Loop)

The HITL Module is responsible for handling cases where the system cannot confidently generate an automated response. When triggered, it routes the query to a human agent for manual resolution. This module ensures reliability and prevents incorrect or misleading answers in critical situations. It may include logging, notification, or ticketing mechanisms to

facilitate human intervention. This component is essential for making the system suitable for real-world customer support environments.



2. DATA STRUCTURES

2.1 Document Representation

The document representation defines how the raw PDF content is stored after being processed by the document loader. Instead of keeping the document as a single block of text, it is typically represented as a structured object containing the extracted text along with metadata such as file name, page number, and source. This structured representation allows the system to track the origin of information and maintain context when retrieving answers. It also enables better traceability, which is important for debugging and improving response accuracy.

2.2 Chunk Format

The chunk format specifies how the document is divided and stored as smaller segments. Each chunk is usually represented as an object containing the chunk text and associated metadata such as chunk ID, source document, and position within the document. In many cases, overlapping text is included between chunks to preserve context. This structure ensures that each chunk is meaningful and self-contained, which improves retrieval quality and ensures that relevant information is not fragmented during processing.

2.3 Embedding Structure

The embedding structure represents how text chunks are converted into numerical vectors. Each embedding is typically stored as a high-dimensional array of floating-point numbers, along with metadata linking it back to its corresponding chunk. These embeddings are stored in a vector database like ChromaDB. This structure enables efficient similarity search by allowing the system to compare vectors mathematically. It is essential for performing semantic retrieval, where meaning is prioritized over exact word matching.

2.4 Query-Response Schema

The query-response schema defines how user queries and system responses are structured within the application. A query object usually includes the user input text, a timestamp, and optionally a session ID. The response object contains the generated answer, references to the

retrieved chunks, and a status indicating whether the response was generated automatically or escalated. This structured approach ensures consistency in communication between modules and helps in logging, debugging, and tracking user interactions.

2.5 State Object for Graph

The state object is a central data structure used by the graph workflow engine to manage the flow of information between different modules. It typically contains fields such as the user query, retrieved context, generated response, intent classification, and routing decision. As the query moves through the workflow, this state object is updated at each stage. This design allows the system to maintain continuity, track progress, and make dynamic decisions based on the current state. It is particularly important in graph-based systems like LangGraph, where multiple nodes interact and share data.

3. WORKFLOW DESIGN (LANG GRAPH)

The workflow of the system is implemented using LangGraph, where the entire application is modeled as a graph. Each step in the query processing pipeline is represented as a node, and the flow between these nodes is controlled using edges. The workflow is state-driven, meaning all nodes operate on a shared state object that carries data throughout the execution.

3.1 Nodes: Processing Node and Output Node

In this system, nodes are implemented as Python functions that take the current state as input and return updates to the state.

Processing Nodes

Processing nodes perform the core logic of the system. In your implementation, the following are processing nodes:

- **Intent Detection Node (intent)**
  This node analyzes the user query and determines whether it should be handled using the RAG pipeline or escalated to a human. It updates the state with the detected intent.

- **RAG Node (rag)**
  This node performs retrieval and response generation. It fetches relevant document chunks from the vector database, constructs a context, and uses the LLM to generate an answer. It updates the state with both the context and the generated response.

Output Node

- **Human Node (human)**
  This node acts as an output node for escalation cases. Instead of processing further, it returns a predefined response indicating that the query requires human assistance. It produces the final output when escalation is needed.

Additionally, the **RAG node also acts as an output node** when the query is successfully answered. Therefore, both the rag and human nodes are defined as finish points in the workflow.

3.2 Edges: Flow Between Nodes

Edges define how the workflow transitions from one node to another. In your implementation, edges are conditional and are defined using:

```
graph.add_conditional_edges(
    "intent",
    route,
    {
        "rag": "rag",
        "escalate": "human"
    }
)
```

The flow works as follows:

- The workflow starts at the **intent node** (entry point).
- Based on the detected intent, the route function determines the next node.
- If the intent is "rag", the flow moves to the RAG node.
- If the intent is "escalate", the flow moves to the Human node.

This conditional routing enables dynamic decision-making and ensures that queries are handled appropriately without unnecessary processing.

3.3 State: Data Flowing Between Nodes

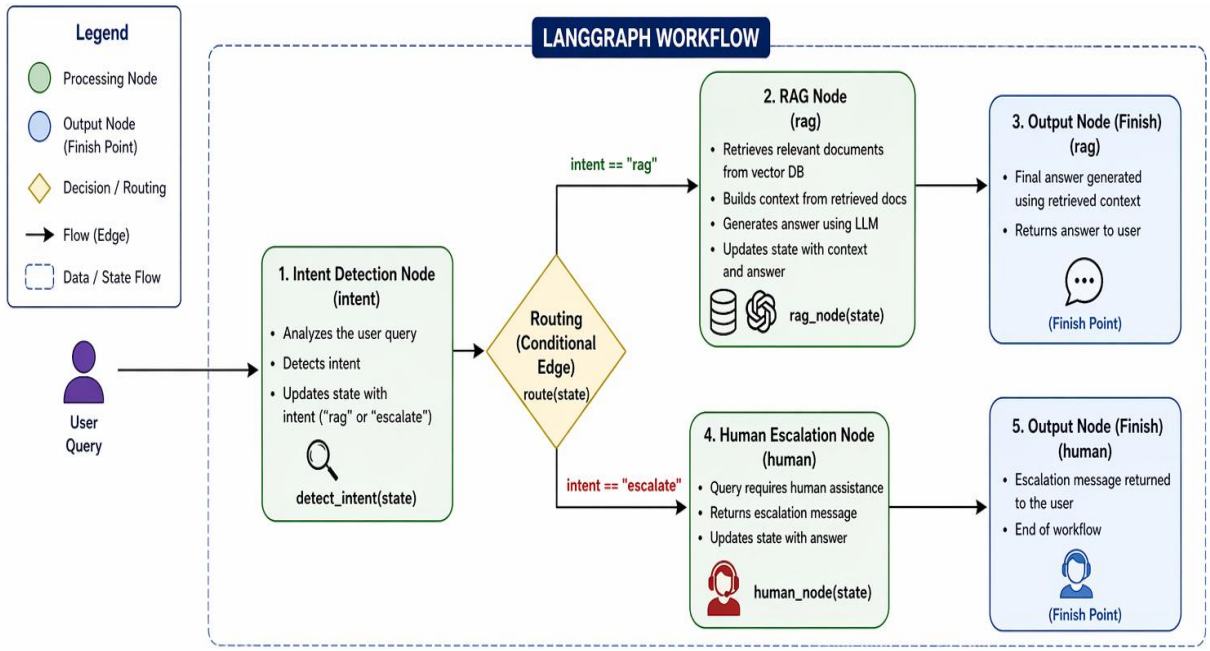
The workflow uses a shared **state object**, defined as:

```
class State(TypedDict):
    query: str
    intent: str
    context: str
    answer: str
```

This state object acts as a central data carrier throughout the workflow. Each node reads from and updates this state:

- The **intent node** updates the intent field
- The **RAG node** updates context and answer
- The **human node** updates the answer field

As the query moves through the graph, the state is progressively enriched with new information. This ensures consistency, traceability, and smooth communication between nodes.



4. CONDITIONAL ROUTING LOGIC

The conditional routing logic is a critical component of the workflow, responsible for deciding how each user query should be processed. It ensures that queries are either handled automatically using the RAG pipeline or escalated to a human when necessary. This decision-making process improves system efficiency, reduces unnecessary computation, and ensures reliable responses in complex scenarios.

4.1 Answer Generation Criteria

A query is processed through the RAG pipeline when it meets the criteria for automated answering. In this case, the system identifies that the query can be resolved using the available knowledge base.

The primary condition for answer generation is when the detected intent is classified as **"rag"**. This indicates that the query is informational, straightforward, and can be answered using retrieved document context. The system then retrieves relevant chunks from the vector database and generates a response using the LLM. This approach ensures that answers are grounded in factual data from the knowledge base and are contextually accurate.

4.2 Escalation Criteria

A query is escalated to the Human-in-the-Loop (HITL) module when the system determines that it cannot confidently or appropriately generate an automated response. This is essential for maintaining reliability and user trust in real-world customer support scenarios. The escalation is triggered when the intent is classified as **"escalate"**, which may occur due to several factors:

4.3 Low Confidence

If the system is uncertain about the correctness of the retrieved information or the generated response, it should avoid providing a potentially incorrect answer. Low confidence may arise from weak similarity scores during retrieval or ambiguous query interpretation. In such cases, escalation ensures that the user receives accurate assistance from a human agent.

4.4 Missing Context

When the retriever fails to find relevant information in the knowledge base, the system lacks sufficient context to generate a meaningful answer. Instead of producing a vague or incorrect response, the query is escalated. This prevents hallucination and maintains the integrity of the system.

4.5 Complex Query

Some queries may involve multi-step reasoning, subjective judgment, or issues beyond the scope of the available documents, such as complaints, refund requests, or requests for human interaction. These queries are classified as complex and are directly routed to the HITL module. This ensures that sensitive or complicated situations are handled appropriately by a human expert.

5. HITL (HUMAN-IN-THE-LOOP) DESIGN

The Human-in-the-Loop (HITL) design is an essential part of the system that ensures reliability and correctness in situations where automated responses may not be sufficient. Instead of forcing the system to answer every query, HITL introduces a mechanism to involve a human agent when needed. This approach improves trust, reduces the risk of incorrect responses, and makes the system suitable for real-world customer support scenarios.

5.1 When Escalation is Triggered

Escalation is triggered when the system determines that a query cannot be confidently handled through the RAG pipeline. In the current implementation, this decision is made during the intent detection stage, where queries containing keywords such as *refund*, *complaint*, *agent*, or *human* are classified under the **"escalate"** intent.

In addition to keyword-based detection, escalation may also occur in the following scenarios:

- When the system has **low confidence** in the retrieved results
- When there is **insufficient or missing context** in the knowledge base
- When the query is **complex or requires human judgment**

These conditions ensure that the system avoids generating unreliable or inappropriate responses.

5.2 What Happens After Escalation

Once a query is classified for escalation, it is routed to the **Human Node** in the workflow. Instead of performing retrieval or response generation, this node returns a predefined message indicating that the query requires human assistance.

In a real-world implementation, this step can be extended to:

- Create a support ticket
- Notify a human agent
- Store the query in a queue or database for further handling

This ensures that the query is properly tracked and handled outside the automated system.

5.3 How Human Response is Integrated

After escalation, the response from the human agent needs to be integrated back into the system to maintain continuity. This can be achieved by updating the system with the human-provided answer and delivering it to the user as the final response.

In more advanced implementations, the system can:

- Log human responses for future reference
- Use resolved queries to improve the knowledge base
- Continuously enhance system performance through feedback

This integration ensures that the system not only resolves the current query but also learns and improves over time.

6. API / INTERFACE DESIGN

The API and interface design define how users interact with the RAG-based Customer Support Assistant and how data is exchanged between the user and the system. It ensures a structured communication mechanism where user queries are processed and responses are returned in a consistent and reliable format. This layer acts as the bridge between the frontend (user interface) and the backend processing pipeline.

6.1 Input Format

The input to the system is a user query submitted through an interface such as a command-line interface, web application, or API request. The query is typically represented in a structured format to ensure consistency and ease of processing.

A standard input format can be defined as a JSON object containing the user's query:

```
{
  "query": "What is the knowledge pdf about?"
}
```

In more advanced implementations, additional fields may be included, such as:

- user_id for tracking user sessions
- timestamp for logging purposes
- session_id for maintaining conversation context

This structured format ensures that the system can easily extract and process the required information.

6.2 Output Format

The output of the system is the response generated after processing the query. It is also returned in a structured format to maintain consistency and support integration with other systems.

A typical output format is:

```
{
  "answer": "The knowledge PDF appears to be about the voting phase of a two-phase commit protocol in a distributed transaction.",
  "intent": "rag"
}
```

For escalated queries, the output may look like:

```
{
  "answer": "This query requires human assistance. Escalating to support team.",
  "intent": "escalate"
}
```

Optionally, the response can include:

- context (retrieved information used for answering)
- status (success or escalated)

This structure ensures clarity in communication and supports debugging and monitoring.

6.3 Interaction Flow

The interaction flow defines how the user query moves through the system and how the response is generated and returned.

1. The user submits a query through the interface.
2. The query is received by the backend API and passed to the graph workflow.
3. The system processes the query starting with intent detection.
4. Based on the detected intent:
    - If the query is suitable for automated answering, it is processed through the RAG pipeline.
    - If the query requires escalation, it is routed to the HITL module.
5. The final response is generated, either automatically or through human intervention.
6. The response is returned to the user in the defined output format.

This interaction flow ensures a seamless and efficient communication process between the user and the system.

7. ERROR HANDLING

7.1 Missing Data

Missing data errors occur when required inputs are not available or are incomplete. This can happen if the user submits an empty query or if the system fails to load the document properly. To handle this, the system validates the input before processing. If the query is empty or invalid, the system returns a clear message prompting the user to provide a valid query. Similarly, if the knowledge base is not loaded or accessible, the system can notify the user or trigger a fallback mechanism. This prevents unnecessary processing and ensures that the system operates on valid data.

7.2 No Relevant Chunks Found

This scenario occurs when the retriever is unable to find any relevant information from the vector database. It may happen if the query is outside the scope of the knowledge base or if the similarity scores are too low. Instead of generating an incorrect or vague response, the system should handle this case explicitly. It can either:

- Return a message indicating that no relevant information was found, or
- Trigger escalation to the Human-in-the-Loop (HITL) module

This approach prevents hallucination by the LLM and maintains the accuracy and trustworthiness of the system.

7.3 LLM Failure

LLM failures can occur due to API errors, timeouts, or unexpected responses. Since the LLM is responsible for generating the final answer, failures at this stage must be handled carefully. The system can implement error handling mechanisms such as:

- Retry logic for temporary failures
- Timeout handling to prevent long delays
- Fallback responses when the LLM is unavailable

In critical cases, the system can escalate the query to the HITL module to ensure that the user still receives a response. Logging these errors is also important for debugging and improving system performance.